Complete SONiC EVPN-VXLAN Tutorial: Architecture, Configuration, and Verification

Table of Contents

1. Introduction to SONiC
2. Architecture Overview
3. Lab Topology
4. Configuration Steps
5. Verification and Show Commands
6. Troubleshooting

---

1. Introduction to SONiC {#introduction}

SONiC (Software for Open Networking in the Cloud) is an open-source network operating system based on Linux that runs on switches from multiple vendors. Originally developed by Microsoft and now maintained by the Linux Foundation, SONiC provides a full-suite layer 2/3 functionality with support for BGP, EVPN, VXLAN, and other modern data center protocols.

Key Components

Architecture Layers:

- **Linux Kernel**: Base operating system (Debian-based)
- **Switch State Service (SwSS)**: Core SONiC component managing switch state
- **ASIC SDK**: Hardware abstraction layer (SAI - Switch Abstraction Interface)
- **Docker Containers**: Each service runs in isolated containers

Critical Containers:

- `database`: Redis database storing all switch state
- `swss`: Switch State Service handling configuration
- `syncd`: Synchronizes state to ASIC
- `bgp`: FRRouting BGP daemon
- `teamd`: LAG management
- `lldp`: Link Layer Discovery Protocol

- `snmp`: SNMP service

Database Architecture

SONiC uses Redis as its central database with multiple sub-databases:

- **APPL_DB**: Application layer database
- **ASIC_DB**: ASIC state database
- **COUNTERS_DB**: Statistics and counters
- **CONFIG_DB**: Configuration database
- **STATE_DB**: State information

---

2. Architecture Overview {#architecture}

EVPN-VXLAN Fundamentals

VXLAN (Virtual Extensible LAN) encapsulates layer 2 Ethernet frames in layer 4 UDP packets, enabling layer 2 overlay networks over layer 3 infrastructure.

EVPN (Ethernet VPN) is a BGP-based control plane for VXLAN that provides:

- MAC address learning via BGP
- ARP/ND suppression
- Multi-tenancy support
- Integrated routing and bridging

SONiC EVPN Components

```
┌──────────────────────────────────────────────┐
│        FRRouting (BGP EVPN)                  │
│  - BGP L2VPN EVPN address family             │
│  - Route types: 2 (MAC/IP), 3 (IMET)         │
└──────────────────────┬───────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────┐
│        VXLAN Manager                         │
│  - VNI management                            │
│  - VTEP configuration                        │
└──────────────────────┬───────────────────────┘
                       │
```

```
┌──────────────────────────▼──────────────────────────┐
│       Kernel VXLAN Interface                         │
│  - Linux VXLAN netdev                                │
│  - Bridge FDB                                        │
└──────────────────────────┬──────────────────────────┘
                           │
┌──────────────────────────▼──────────────────────────┐
│       Hardware (ASIC)                                │
│  - Encap/Decap in hardware                           │
└──────────────────────────────────────────────────────┘
```

---

3. Lab Topology {#topology}

Network Design

We'll configure a Clos network with:

- 2 Spine switches (Spine1, Spine2)
- 3 Leaf switches (Leaf1, Leaf2, Leaf3)
- BGP underlay with EVPN overlay
- 2 VLANs/VNIs distributed across leaves

```
          ┌──────────────┐      ┌──────────────┐
          │  Spine1  │       │  Spine2  │
          │ 10.0.0.1 │       │ 10.0.0.2 │
          └──────┬┬┬┬──────┘      └──┬┬┬┬──────┘
            │ │ │        │ │ │
    ┌──────────────────┘ │ └──────────────────┘ │ └──────────────────┐
    │          │          │          │
    │          └──────────────────────┘          │
    │              │              │
  ┌──────▼──────┐        ┌──────▼──────┐        ┌──────▼──────┐
  │  Leaf1  │          │  Leaf2  │          │  Leaf3  │
  │10.0.0.11│          │10.0.0.12│          │10.0.0.13│
  └──────┬──────┘        └──────┬──────┘        └──────┬──────┘
    │              │              │
  [Servers in            [Servers in         [Servers in
   VLAN 10/20]            VLAN 10/20]         VLAN 10/20]
```

IP Addressing Plan

Loopbacks (for VTEP):

- Spine1: 10.0.0.1/32
- Spine2: 10.0.0.2/32
- Leaf1: 10.0.0.11/32
- Leaf2: 10.0.0.12/32
- Leaf3: 10.0.0.13/32

Point-to-Point Links:

- Spine1 to Leaf1: 10.1.1.0/31 (Spine: .0, Leaf: .1)
- Spine1 to Leaf2: 10.1.1.2/31 (Spine: .2, Leaf: .3)
- Spine1 to Leaf3: 10.1.1.4/31 (Spine: .4, Leaf: .5)
- Spine2 to Leaf1: 10.1.2.0/31 (Spine: .0, Leaf: .1)
- Spine2 to Leaf2: 10.1.2.2/31 (Spine: .2, Leaf: .3)
- Spine2 to Leaf3: 10.1.2.4/31 (Spine: .4, Leaf: .5)

VLANs/VNIs:

- VLAN 10 → VNI 10010
- VLAN 20 → VNI 10020

BGP:

- AS 65000 (Spines)
- AS 65001 (Leaf1)
- AS 65002 (Leaf2)
- AS 65003 (Leaf3)

---

4. Configuration Steps {#configuration}

4.1 Initial System Configuration

Accessing SONiC CLI

```
bash
```

```
# Login to switch
ssh admin@<switch-ip>

# Enter privileged mode
admin@sonic:~$ sudo -i

# Access SONiC configuration
root@sonic:~# sonic-cli
```

4.2 Spine1 Configuration

Configure Hostname and Loopback

```bash
# Enter configuration mode
sonic# configure terminal

# Set hostname
sonic(config)# hostname Spine1

# Configure loopback
Spine1(config)# interface Loopback 0
Spine1(config-if-Loopback0)# ip address 10.0.0.1/32
Spine1(config-if-Loopback0)# exit
```

Configure Physical Interfaces

```bash
```

```
# Interface to Leaf1
Spine1(config)# interface Ethernet0
Spine1(config-if-Ethernet0)# no switchport
Spine1(config-if-Ethernet0)# mtu 9216
Spine1(config-if-Ethernet0)# ip address 10.1.1.0/31
Spine1(config-if-Ethernet0)# exit

# Interface to Leaf2
Spine1(config)# interface Ethernet4
Spine1(config-if-Ethernet4)# no switchport
Spine1(config-if-Ethernet4)# mtu 9216
Spine1(config-if-Ethernet4)# ip address 10.1.1.2/31
Spine1(config-if-Ethernet4)# exit

# Interface to Leaf3
Spine1(config)# interface Ethernet8
Spine1(config-if-Ethernet8)# no switchport
Spine1(config-if-Ethernet8)# mtu 9216
Spine1(config-if-Ethernet8)# ip address 10.1.1.4/31
Spine1(config-if-Ethernet8)# exit
```

Configure BGP for Underlay

```bash
Spine1(config)# router bgp 65000
Spine1(config-router-bgp)# router-id 10.0.0.1
Spine1(config-router-bgp)# neighbor 10.1.1.1 remote-as 65001
Spine1(config-router-bgp)# neighbor 10.1.1.3 remote-as 65002
Spine1(config-router-bgp)# neighbor 10.1.1.5 remote-as 65003
Spine1(config-router-bgp)# address-family ipv4 unicast
Spine1(config-router-bgp-af)# network 10.0.0.1/32
Spine1(config-router-bgp-af)# exit
Spine1(config-router-bgp)# exit
```

Configure BGP EVPN

```bash
```

```
Spine1(config)# router bgp 65000
Spine1(config-router-bgp)# neighbor 10.1.1.1 remote-as 65001
Spine1(config-router-bgp)# neighbor 10.1.1.3 remote-as 65002
Spine1(config-router-bgp)# neighbor 10.1.1.5 remote-as 65003
Spine1(config-router-bgp)# address-family l2vpn evpn
Spine1(config-router-bgp-af)# neighbor 10.1.1.1 activate
Spine1(config-router-bgp-af)# neighbor 10.1.1.3 activate
Spine1(config-router-bgp-af)# neighbor 10.1.1.5 activate
Spine1(config-router-bgp-af)# exit
Spine1(config-router-bgp)# exit
Spine1(config)# exit
```

4.3 Spine2 Configuration

Similar to Spine1 but with adjusted IPs:

```bash
```

```
Spine2(config)# hostname Spine2
Spine2(config)# interface Loopback 0
Spine2(config-if-Loopback0)# ip address 10.0.0.2/32

Spine2(config)# interface Ethernet0
Spine2(config-if-Ethernet0)# no switchport
Spine2(config-if-Ethernet0)# mtu 9216
Spine2(config-if-Ethernet0)# ip address 10.1.2.0/31

Spine2(config)# interface Ethernet4
Spine2(config-if-Ethernet4)# no switchport
Spine2(config-if-Ethernet4)# mtu 9216
Spine2(config-if-Ethernet4)# ip address 10.1.2.2/31

Spine2(config)# interface Ethernet8
Spine2(config-if-Ethernet8)# no switchport
Spine2(config-if-Ethernet8)# mtu 9216
Spine2(config-if-Ethernet8)# ip address 10.1.2.4/31

Spine2(config)# router bgp 65000
Spine2(config-router-bgp)# router-id 10.0.0.2
Spine2(config-router-bgp)# neighbor 10.1.2.1 remote-as 65001
Spine2(config-router-bgp)# neighbor 10.1.2.3 remote-as 65002
Spine2(config-router-bgp)# neighbor 10.1.2.5 remote-as 65003
Spine2(config-router-bgp)# address-family ipv4 unicast
Spine2(config-router-bgp-af)# network 10.0.0.2/32
Spine2(config-router-bgp-af)# exit
Spine2(config-router-bgp)# address-family l2vpn evpn
Spine2(config-router-bgp-af)# neighbor 10.1.2.1 activate
Spine2(config-router-bgp-af)# neighbor 10.1.2.3 activate
Spine2(config-router-bgp-af)# neighbor 10.1.2.5 activate
Spine2(config-router-bgp-af)# exit
```

4.4 Leaf1 Configuration (VTEP)

Basic Configuration

```bash
```

```
Leaf1(config)# hostname Leaf1
Leaf1(config)# interface Loopback 0
Leaf1(config-if-Loopback0)# ip address 10.0.0.11/32

# Uplinks to Spines
Leaf1(config)# interface Ethernet0
Leaf1(config-if-Ethernet0)# no switchport
Leaf1(config-if-Ethernet0)# mtu 9216
Leaf1(config-if-Ethernet0)# ip address 10.1.1.1/31

Leaf1(config)# interface Ethernet4
Leaf1(config-if-Ethernet4)# no switchport
Leaf1(config-if-Ethernet4)# mtu 9216
Leaf1(config-if-Ethernet4)# ip address 10.1.2.1/31
```

Access Port Configuration

```bash
# Server-facing ports in VLAN 10
Leaf1(config)# interface Ethernet16
Leaf1(config-if-Ethernet16)# switchport access vlan 10

# Server-facing ports in VLAN 20
Leaf1(config)# interface Ethernet20
Leaf1(config-if-Ethernet20)# switchport access vlan 20
```

BGP Underlay Configuration

```bash
Leaf1(config)# router bgp 65001
Leaf1(config-router-bgp)# router-id 10.0.0.11
Leaf1(config-router-bgp)# neighbor 10.1.1.0 remote-as 65000
Leaf1(config-router-bgp)# neighbor 10.1.2.0 remote-as 65000
Leaf1(config-router-bgp)# address-family ipv4 unicast
Leaf1(config-router-bgp-af)# network 10.0.0.11/32
Leaf1(config-router-bgp-af)# exit
```

VXLAN Tunnel Configuration

```bash
```

```
Leaf1(config)# interface vxlan vtep1
Leaf1(config-if-vxlan-vtep1)# source-ip 10.0.0.11
Leaf1(config-if-vxlan-vtep1)# map vlan 10 vni 10010
Leaf1(config-if-vxlan-vtep1)# map vlan 20 vni 10020
Leaf1(config-if-vxlan-vtep1)# exit
```

BGP EVPN Configuration

```bash
Leaf1(config)# router bgp 65001
Leaf1(config-router-bgp)# address-family l2vpn evpn
Leaf1(config-router-bgp-af)# neighbor 10.1.1.0 activate
Leaf1(config-router-bgp-af)# neighbor 10.1.2.0 activate
Leaf1(config-router-bgp-af)# advertise-all-vni
Leaf1(config-router-bgp-af)# exit
```

4.5 Leaf2 Configuration

```bash
```

```
Leaf2(config)# hostname Leaf2
Leaf2(config)# interface Loopback 0
Leaf2(config-if-Loopback0)# ip address 10.0.0.12/32

Leaf2(config)# interface Ethernet0
Leaf2(config-if-Ethernet0)# no switchport
Leaf2(config-if-Ethernet0)# mtu 9216
Leaf2(config-if-Ethernet0)# ip address 10.1.1.3/31

Leaf2(config)# interface Ethernet4
Leaf2(config-if-Ethernet4)# no switchport
Leaf2(config-if-Ethernet4)# mtu 9216
Leaf2(config-if-Ethernet4)# ip address 10.1.2.3/31

Leaf2(config)# interface Ethernet16
Leaf2(config-if-Ethernet16)# switchport access vlan 10

Leaf2(config)# interface Ethernet20
Leaf2(config-if-Ethernet20)# switchport access vlan 20

Leaf2(config)# router bgp 65002
Leaf2(config-router-bgp)# router-id 10.0.0.12
Leaf2(config-router-bgp)# neighbor 10.1.1.2 remote-as 65000
Leaf2(config-router-bgp)# neighbor 10.1.2.2 remote-as 65000
Leaf2(config-router-bgp)# address-family ipv4 unicast
Leaf2(config-router-bgp-af)# network 10.0.0.12/32
Leaf2(config-router-bgp-af)# exit

Leaf2(config)# interface vxlan vtep1
Leaf2(config-if-vxlan-vtep1)# source-ip 10.0.0.12
Leaf2(config-if-vxlan-vtep1)# map vlan 10 vni 10010
Leaf2(config-if-vxlan-vtep1)# map vlan 20 vni 10020

Leaf2(config)# router bgp 65002
Leaf2(config-router-bgp)# address-family l2vpn evpn
Leaf2(config-router-bgp-af)# neighbor 10.1.1.2 activate
Leaf2(config-router-bgp-af)# neighbor 10.1.2.2 activate
Leaf2(config-router-bgp-af)# advertise-all-vni
```

4.6 Leaf3 Configuration

```bash
```

```
Leaf3(config)# hostname Leaf3
Leaf3(config)# interface Loopback 0
Leaf3(config-if-Loopback0)# ip address 10.0.0.13/32

Leaf3(config)# interface Ethernet0
Leaf3(config-if-Ethernet0)# no switchport
Leaf3(config-if-Ethernet0)# mtu 9216
Leaf3(config-if-Ethernet0)# ip address 10.1.1.5/31

Leaf3(config)# interface Ethernet4
Leaf3(config-if-Ethernet4)# no switchport
Leaf3(config-if-Ethernet4)# mtu 9216
Leaf3(config-if-Ethernet4)# ip address 10.1.2.5/31

Leaf3(config)# interface Ethernet16
Leaf3(config-if-Ethernet16)# switchport access vlan 10

Leaf3(config)# interface Ethernet20
Leaf3(config-if-Ethernet20)# switchport access vlan 20

Leaf3(config)# router bgp 65003
Leaf3(config-router-bgp)# router-id 10.0.0.13
Leaf3(config-router-bgp)# neighbor 10.1.1.4 remote-as 65000
Leaf3(config-router-bgp)# neighbor 10.1.2.4 remote-as 65000
Leaf3(config-router-bgp)# address-family ipv4 unicast
Leaf3(config-router-bgp-af)# network 10.0.0.13/32
Leaf3(config-router-bgp-af)# exit

Leaf3(config)# interface vxlan vtep1
Leaf3(config-if-vxlan-vtep1)# source-ip 10.0.0.13
Leaf3(config-if-vxlan-vtep1)# map vlan 10 vni 10010
Leaf3(config-if-vxlan-vtep1)# map vlan 20 vni 10020

Leaf3(config)# router bgp 65003
Leaf3(config-router-bgp)# address-family l2vpn evpn
Leaf3(config-router-bgp-af)# neighbor 10.1.1.4 activate
Leaf3(config-router-bgp-af)# neighbor 10.1.2.4 activate
Leaf3(config-router-bgp-af)# advertise-all-vni
```

5. Verification and Show Commands {#verification}

5.1 Verify BGP Underlay (from Spine1)

```bash
Spine1# show ip bgp summary
```

Output:

```
BGP router identifier 10.0.0.1, local AS number 65000
RIB entries 9, using 1728 bytes of memory
Peers 3, using 64 KiB of memory

Neighbor        V    AS MsgRcvd MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd
10.1.1.1        4 65001     145     148        0    0    0 02:18:42            1
10.1.1.3        4 65002     143     146        0    0    0 02:17:35            1
10.1.1.5        4 65003     141     144        0    0    0 02:16:28            1

Total number of neighbors 3
```

Explanation:

- All three leaf switches are BGP neighbors
- Each leaf is advertising 1 prefix (their loopback)
- Sessions are established (State shows prefix count, not "Idle" or "Active")
- MsgRcvd/MsgSent show active communication

5.2 Verify IP Routes (from Spine1)

```bash
Spine1# show ip route
```

Output:

```
Codes: K - kernel route, C - connected, S - static, R - RIP,
       O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, D - SHARP,
       F - PBR, f - OpenFabric,
       > - selected route, * - FIB route, q - queued route, r - rejected route

C>* 10.0.0.1/32 is directly connected, Loopback0, 02:35:12
B>* 10.0.0.11/32 [20/0] via 10.1.1.1, Ethernet0, 02:19:53
B>* 10.0.0.12/32 [20/0] via 10.1.1.3, Ethernet4, 02:18:46
B>* 10.0.0.13/32 [20/0] via 10.1.1.5, Ethernet8, 02:17:39
C>* 10.1.1.0/31 is directly connected, Ethernet0, 02:35:08
C>* 10.1.1.2/31 is directly connected, Ethernet4, 02:35:08
C>* 10.1.1.4/31 is directly connected, Ethernet8, 02:35:08
```

Explanation:

- C routes are directly connected
- B routes are learned via BGP
- All leaf loopbacks are reachable via BGP
- This confirms IP underlay connectivity

5.3 Verify BGP EVPN (from Leaf1)

```bash
Leaf1# show bgp l2vpn evpn summary
```

Output:

```
BGP router identifier 10.0.0.11, local AS number 65001
vrf-id 0
BGP table version 0
RIB entries 13, using 2496 bytes of memory
Peers 2, using 43 KiB of memory

Neighbor        V    AS MsgRcvd MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd
10.1.1.0        4 65000     156     159        0    0    0 02:21:34            6
10.1.2.0        4 65000     154     157        0    0    0 02:20:27            6

Total number of neighbors 2
```

Explanation:

- Leaf1 has two EVPN peerings (to both spines)
- Each spine is reflecting 6 EVPN routes from other leaves
- State/PfxRcd showing numbers confirms EVPN session is up

5.4 View EVPN Routes (from Leaf1)

```bash
Leaf1# show bgp l2vpn evpn
```

Output:

```
BGP table version is 7, local router ID is 10.0.0.11
Status codes: s suppressed, d damped, h history, * valid, > best, i - internal
Origin codes: i - IGP, e - EGP, ? - incomplete
EVPN type-1 prefix: [1]:[ESI]:[EthTag]:[IPlen]:[VTEP-IP]
EVPN type-2 prefix: [2]:[EthTag]:[MAClen]:[MAC]:[IPlen]:[IP]
EVPN type-3 prefix: [3]:[EthTag]:[IPlen]:[OrigIP]
EVPN type-4 prefix: [4]:[ESI]:[IPlen]:[OrigIP]
EVPN type-5 prefix: [5]:[EthTag]:[IPlen]:[IP]

   Network          Next Hop            Metric LocPrf Weight Path
                    Extended Community
Route Distinguisher: 10.0.0.11:2
*> [3]:[0]:[32]:[10.0.0.11]
                    10.0.0.11                      32768 i
                    ET:8 RT:65001:10010
*> [3]:[0]:[32]:[10.0.0.11]
                    10.0.0.11                      32768 i
                    ET:8 RT:65001:10020
Route Distinguisher: 10.0.0.12:2
*  [3]:[0]:[32]:[10.0.0.12]
                    10.0.0.12                        0 65000 65002 i
                    RT:65002:10010 ET:8
*> [3]:[0]:[32]:[10.0.0.12]
                    10.0.0.12                        0 65000 65002 i
                    RT:65002:10010 ET:8
*  [3]:[0]:[32]:[10.0.0.12]
                    10.0.0.12                        0 65000 65002 i
                    RT:65002:10020 ET:8
*> [3]:[0]:[32]:[10.0.0.12]
                    10.0.0.12                        0 65000 65002 i
                    RT:65002:10020 ET:8
Route Distinguisher: 10.0.0.13:2
*  [3]:[0]:[32]:[10.0.0.13]
                    10.0.0.13                        0 65000 65003 i
                    RT:65003:10010 ET:8
*> [3]:[0]:[32]:[10.0.0.13]
                    10.0.0.13                        0 65000 65003 i
                    RT:65003:10010 ET:8
*  [3]:[0]:[32]:[10.0.0.13]
                    10.0.0.13                        0 65000 65003 i
                    RT:65003:10020 ET:8
*> [3]:[0]:[32]:[10.0.0.13]
```

```
                10.0.0.13                    0 65000 65003 i
                RT:65003:10020 ET:8
```

Explanation:

- Type-3 routes are IMET (Inclusive Multicast Ethernet Tag) routes
- These advertise VTEP endpoints for BUM traffic
- Each leaf advertises two Type-3 routes (one per VNI: 10010, 10020)
- RT (Route Target) helps import/export routes to correct VRFs/VNIs
- ET:8 indicates Encapsulation Type is VXLAN
- Duplicate routes (*) show paths through both spines (ECMP)

5.5 Verify VXLAN Tunnel Status (from Leaf1)

```bash
Leaf1# show vxlan tunnel
```

Output:

```
VTEP Name      : vtep1
SIP            : 10.0.0.11
Creation Source: EVPN

DIP           Status    Source    McastGrp    Count
10.0.0.12       Up        EVPN      N/A        2
10.0.0.13       Up        EVPN      N/A        2
```

Explanation:

- VTEP source IP is Leaf1's loopback (10.0.0.11)
- Two destination VTEPs discovered via EVPN
- DIP 10.0.0.12 = Leaf2, DIP 10.0.0.13 = Leaf3
- Status "Up" means tunnels are operational
- Count indicates number of VNIs using this tunnel

5.6 Verify VXLAN VNI Mapping (from Leaf1)

```bash
```

```
Leaf1# show vxlan vlanvnimap
```

Output:

```
+---------+-------+
| VLAN    | VNI   |
+---------+-------+
| Vlan10  | 10010 |
| Vlan20  | 10020 |
+---------+-------+
Total count: 2
```

Explanation:

- VLAN 10 is mapped to VNI 10010
- VLAN 20 is mapped to VNI 10020
- These mappings enable layer 2 extension across the fabric

5.7 Verify VXLAN Remote MAC Addresses (from Leaf1)

After some traffic flows, check learned MAC addresses:

```bash
Leaf1# show vxlan remotemac all
```

Output:

```
+-------+-------------------+--------------+-------+
| VLAN  | MAC               | RemoteVTEP   | VNI   |
+-------+-------------------+--------------+-------+
| Vlan10| 00:11:22:33:44:55 | 10.0.0.12    | 10010 |
| Vlan10| 00:11:22:33:44:66 | 10.0.0.13    | 10010 |
| Vlan20| 00:aa:bb:cc:dd:ee | 10.0.0.12    | 10020 |
| Vlan20| 00:aa:bb:cc:dd:ff | 10.0.0.13    | 10020 |
+-------+-------------------+--------------+-------+
Total count: 4
```

Explanation:

- MAC addresses learned via EVPN Type-2 routes

- RemoteVTEP shows which leaf the MAC is behind
- VNI indicates which overlay network the MAC belongs to
- These entries enable efficient unicast forwarding without flooding

5.8 View Local MAC Address Table (from Leaf1)

```bash
Leaf1# show mac address-table
```

Output:

```
Vlan    MAC Address        Type       Port          Age(s)
----    -----------        ----       ----          ------
10      00:50:56:11:11:11  DYNAMIC    Ethernet16    245
20      00:50:56:22:22:22  DYNAMIC    Ethernet20    180
10      00:11:22:33:44:55  EVPN       vtep1         -
10      00:11:22:33:44:66  EVPN       vtep1         -
20      00:aa:bb:cc:dd:ee  EVPN       vtep1         -
20      00:aa:bb:cc:dd:ff  EVPN       vtep1         -
Total entries: 6
```

Explanation:

- Local MACs (00:50:56:xx) learned on physical ports
- Remote MACs learned via EVPN, forwarded to vtep1 tunnel
- Type "EVPN" indicates MAC learned via BGP EVPN
- Age shows seconds since last traffic (EVPN MACs don't age out)

5.9 Verify Interface Status (from Leaf1)

```bash
Leaf1# show interface status
```

Output:

```
Interface       Status    VLAN    Speed    Type                Asymmetric PFC
---------------  --------  ------  -------  -------------------  ---------------
Ethernet0        up        routed  100G     QSFP28 or later      N/A
Ethernet4        up        routed  100G     QSFP28 or later      N/A
Ethernet16       up        10      10G      SFP+                 N/A
Ethernet20       up        20      10G      SFP+                 N/A
Loopback0        up        routed  N/A      N/A                  N/A
```

Explanation:

- Ethernet0/4 are routed uplink ports (no switchport)
- Ethernet16/20 are access ports in VLANs 10 and 20
- All interfaces operationally "up"

5.10 Deep Dive: EVPN Route Details (from Leaf1)

```bash
Leaf1# show bgp l2vpn evpn route rd 10.0.0.12:2
```

Output:

```
BGP routing table entry for 10.0.0.12:2:[3]:[0]:[32]:[10.0.0.12]
Paths: (2 available, best #2)
  Advertised to non peer-group peers:
  10.1.1.0 10.1.2.0
  65000 65002
    10.0.0.12 from 10.1.2.0 (10.0.0.2)
      Origin IGP, valid, external, best
      Extended Community: RT:65002:10010 ET:8
      Last update: Mon Jan 13 14:23:45 2025
  65000 65002
    10.0.0.12 from 10.1.1.0 (10.0.0.1)
      Origin IGP, valid, external
      Extended Community: RT:65002:10010 ET:8
      Last update: Mon Jan 13 14:23:45 2025
```

Explanation:

- RD (Route Distinguisher) 10.0.0.12:2 identifies routes from Leaf2
- Type-3 route advertises Leaf2's VTEP (10.0.0.12)

- Two paths available (via Spine1 and Spine2) - ECMP capability
- RT:65002:10010 is Route Target for VNI 10010
- ET:8 confirms VXLAN encapsulation

5.11 Verify FRRouting BGP Process (Linux Level)

From the Linux shell:

```bash
admin@Leaf1:~$ sudo vtysh -c "show bgp summary"
```

Output:

```
IPv4 Unicast Summary:
BGP router identifier 10.0.0.11, local AS number 65001 vrf-id 0
BGP table version 8
RIB entries 9, using 1728 bytes of memory
Peers 2, using 43 KiB of memory

Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down  State/PfxRcd
10.1.1.0        4      65000       178       181        0    0    0 02:35:12        4
10.1.2.0        4      65000       176       179        0    0    0 02:34:05        4

L2VPN EVPN Summary:
BGP router identifier 10.0.0.11, local AS number 65001 vrf-id 0
BGP table version 0
RIB entries 13, using 2496 bytes of memory
Peers 2, using 43 KiB of memory

Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down  State/PfxRcd
10.1.1.0        4      65000       178       181        0    0    0 02:35:12        6
10.1.2.0        4      65000       176       179        0    0    0 02:34:05        6
```

Explanation:

- FRRouting handles both IPv4 unicast and L2VPN EVPN families
- Same BGP session carries both address families
- Message counts include keepalives and updates
- PfxRcd shows received prefix count per address family

5.12 Check Redis Database Entries

```bash
admin@Leaf1:~$ redis-cli -n 4 KEYS "*VXLAN*"
```

Output:

```
1) "VXLAN_TUNNEL_MAP|vtep1|map_10010_Vlan10"
2) "VXLAN_TUNNEL_MAP|vtep1|map_10020_Vlan20"
3) "VXLAN_TUNNEL|vtep1"
```

Explanation:

- SONiC stores VXLAN config in CONFIG_DB (database 4)
- VXLAN_TUNNEL stores tunnel parameters
- VXLAN_TUNNEL_MAP stores VLAN to VNI mappings
- This data is consumed by swss containers

5.13 View VXLAN Tunnel in Kernel

```bash
admin@Leaf1:~$ ip -d link show type vxlan
```

Output:

```
55: vtep1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9100 qdisc noqueue master Bridge state UP
    link/ether 1a:2b:3c:4d:5e:6f brd ff:ff:ff:ff:ff:ff promiscuity 1
    vxlan id 0 local 10.0.0.11 dev Loopback0 srcport 0 0 dstport 4789
    nolearning ageing 300 udpcsum noudp6zerocsumtx noudp6zerocsumrx
    bridge_slave state forwarding priority 32 cost 100 hairpin off guard off
    root_block off fastleave off learning off flood on port_id 0x8001 port_no 0x1
```

Explanation:

- vtep1 is the VXLAN interface created in Linux kernel
- Source IP: 10.0.0.11 (Leaf1's loopback)
- UDP destination port: 4789 (standard VXLAN port)
- Bridge master indicates it's part of the bridge domain

- nolearning flag means MAC learning happens via control plane (EVPN)

5.14 View Bridge FDB (Forwarding Database)

```bash
admin@Leaf1:~$ bridge fdb show dev vtep1
```

Output:

```
00:00:00:00:00:00 dst 10.0.0.12 self permanent
00:00:00:00:00:00 dst 10.0.0.13 self permanent
00:11:22:33:44:55 dst 10.0.0.12 self
00:11:22:33:44:66 dst 10.0.0.13 self
00:aa:bb:cc:dd:ee dst 10.0.0.12 self
00:aa:bb:cc:dd:ff dst 10.0.0.13 self
```

Explanation:

- First two entries (00:00:00:00:00:00) are for BUM traffic
- Subsequent entries map remote MACs to destination VTEPs
- "self" flag indicates entry is for the VXLAN device itself
- These entries are programmed by vxlanmgr based on EVPN routes

5.15 Check ASIC Programming (Broadcom Example)

```bash
admin@Leaf1:~$ bcmcmd "l2 show"
```

Output:

```
mac=00:50:56:11:11:11 vlan=10 port=xe0 Static
mac=00:50:56:22:22:22 vlan=20 port=xe4 Static
mac=00:11:22:33:44:55 vlan=10 VXLAN_Tunnel=0x5000001 DIP=10.0.0.12
mac=00:11:22:33:44:66 vlan=10 VXLAN_Tunnel=0x5000002 DIP=10.0.0.13
mac=00:aa:bb:cc:dd:ee vlan=20 VXLAN_Tunnel=0x5000001 DIP=10.0.0.12
mac=00:aa:bb:cc:dd:ff vlan=20 VXLAN_Tunnel=0x5000002 DIP=10.0.0.13
```

Explanation:

- Hardware L2 table shows MAC-to-port/tunnel mappings

- Local MACs point to physical ports (xe0, xe4)
- Remote MACs point to VXLAN tunnels with destination IPs
- syncd daemon synchronizes this from APPL_DB to ASIC_DB

5.16 View Interface Packet Statistics

```bash
Leaf1# show interface counters
```

Output:

```
Interface     RX_OK     RX_BPS    RX_UTIL   RX_ERR    TX_OK     TX_BPS    TX_UTIL   TX_ERR
-----------  --------  ----------  ---------  --------  --------  ----------  ---------  --------
Ethernet0    12458934   45.2 MB/s    4.52%        0  11234567   38.9 MB/s    3.89%        0
Ethernet4    12446821   44.8 MB/s    4.48%        0  11223344   38.5 MB/s    3.85%        0
Ethernet16    8934521   12.3 MB/s    1.23%        0   7823412   10.8 MB/s    1.08%        0
Ethernet20    6745234    8.9 MB/s    0.89%        0   5634123    7.4 MB/s    0.74%        0
```

Explanation:

- RX/TX packet counts show traffic flowing
- BPS (Bytes Per Second) shows throughput
- Zero errors indicate healthy links
- Uplinks (Eth0/4) carry more traffic than access ports

5.17 Check Container Status

```bash
admin@Leaf1:~$ docker ps
```

Output:

```
CONTAINER ID   IMAGE                          STATUS        NAMES
a1b2c3d4e5f6   docker-sonic-mgmt-framework       Up 3 hours    mgmt-framework
b2c3d4e5f6a7   docker-router-advertiser          Up 3 hours    radv
c3d4e5f6a7b8   docker-fpm-frr                   Up 3 hours    bgp
d4e5f6a7b8c9   docker-syncd-brcm                Up 3 hours    syncd
e5f6a7b8c9d0   docker-teamd                     Up 3 hours    teamd
f6a7b8c9d0e1   docker-lldp                      Up 3 hours    lldp
a7b8c9d0e1f2   docker-database                  Up 3 hours    database
b8c9d0e1f2a3   docker-platform-monitor          Up 3 hours    pmon
```

Explanation:

- Each SONiC feature runs in isolated Docker container
- "bgp" container runs FRRouting for BGP/EVPN
- "syncd" synchronizes state to hardware ASIC
- "database" runs Redis storing all configuration
- All containers "Up" indicates healthy system

5.18 Monitor BGP Neighbor Status

```bash
Leaf1# show ip bgp neighbors 10.1.1.0
```

Output:

```
BGP neighbor is 10.1.1.0, remote AS 65000, local AS 65001, external link
  BGP version 4, remote router ID 10.0.0.1
  BGP state = Established, up for 02:42:18
  Last read 00:00:03, Last write 00:00:05
  Hold time is 9, keepalive interval is 3 seconds
  Neighbor capabilities:
    4 Byte AS: advertised and received
    AddPath:
      IPv4 Unicast: RX advertised
      L2VPN EVPN: RX advertised
    Route refresh: advertised and received
    Address families by peer:
      IPv4 Unicast
      L2VPN EVPN
  Message statistics:
    Inq depth is 0
    Outq depth is 0
                         Sent       Rcvd
    Opens:                  1          1
    Notifications:          0          0
    Updates:               12         23
    Keepalives:           512        509
    Route Refresh:          0          0
    Capability:             0          0
    Total:                525        533
```

Explanation:

- BGP state "Established" = session up and running
- Both IPv4 Unicast and L2VPN EVPN address families active
- Keepalive timer: 3 seconds (hold time: 9 seconds)
- Update messages show route exchanges
- No notifications = no errors/resets

5.19 Verify ECMP for VXLAN Traffic

```bash
Leaf1# show ip route 10.0.0.12
```

Output:

```
Routing entry for 10.0.0.12/32
  Known via "bgp", distance 20, metric 0, best
  Last update 02:45:23 ago
  * 10.1.1.0, via Ethernet0, weight 1
  * 10.1.2.0, via Ethernet4, weight 1
```

Explanation:

- Two equal-cost paths to Leaf2's loopback
- Traffic will load-balance across both spines
- Provides redundancy and increased bandwidth
- Hash-based ECMP ensures flow affinity

5.20 Check VXLAN Encapsulation Statistics

```bash
admin@Leaf1:~$ sudo vtysh -c "show evpn vni 10010"
```

Output:

```
VNI: 10010
  Type: L2
  Tenant VRF: default
  RD: 10.0.0.11:2
  Originator IP: 10.0.0.11
  Advertise gateway MAC: Disabled
  Advertise SVI MAC: Disabled
  Advertise VNI: Enabled
  Mcast group: 0.0.0.0
  VxLAN interface: vtep1
  VxLAN ifIndex: 55
  Local VTEPs:
  Remote VTEPs for this VNI:
    10.0.0.12
    10.0.0.13
  Number of MACs (local and remote) known for this VNI: 3
  Number of ARPs (IPv4 and IPv6, local and remote) known for this VNI: 0
```

Explanation:

- VNI 10010 is a Layer 2 VNI
- Advertise VNI enabled means Type-3 routes are sent
- Two remote VTEPs discovered (Leaf2 and Leaf3)
- 3 MAC addresses known (1 local, 2 remote)
- ARP suppression not configured (0 ARP entries)

---

6. Troubleshooting {#troubleshooting}

6.1 BGP Session Not Establishing

Symptom:

```bash
Leaf1# show bgp l2vpn evpn summary
Neighbor        V    AS MsgRcvd MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd
10.1.1.0        4 65000       0       0        0    0    0    never    Active
```

Diagnosis Steps:

```bash
# 1. Verify IP connectivity
Leaf1# ping 10.1.1.0
PING 10.1.1.0 (10.1.1.0): 56 data bytes
Request timeout
--- 10.1.1.0 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss

# 2. Check interface status
Leaf1# show interface Ethernet0
Ethernet0 is down, line protocol is down

# 3. Verify MTU configuration
Leaf1# show interface Ethernet0 | include MTU
MTU 1500 bytes

# 4. Check BGP configuration
Leaf1# show running-config router bgp
```

Common Fixes:

- Interface administratively down: `no shutdown` on interface
- Wrong AS number in neighbor configuration
- MTU mismatch: ensure MTU ≥ 9000 for VXLAN overhead
- Firewall blocking TCP port 179
- IP address mismatch on point-to-point links

6.2 EVPN Routes Not Being Advertised

Symptom:

```bash
Leaf1# show bgp l2vpn evpn
% No BGP prefixes displayed, 0 exist
```

Diagnosis:

```bash
# 1. Check if EVPN address family is activated
Leaf1# show running-config router bgp
router bgp 65001
 neighbor 10.1.1.0 remote-as 65000
 !
 address-family l2vpn evpn
  ! Missing: neighbor 10.1.1.0 activate

# 2. Verify advertise-all-vni is configured
Leaf1# show running-config router bgp | include advertise-all-vni
  (should show: advertise-all-vni)

# 3. Check VXLAN tunnel status
Leaf1# show vxlan tunnel
(should show tunnel configuration)
```

Fix:

```bash
```

```
Leaf1(config)# router bgp 65001
Leaf1(config-router-bgp)# address-family l2vpn evpn
Leaf1(config-router-bgp-af)# neighbor 10.1.1.0 activate
Leaf1(config-router-bgp-af)# advertise-all-vni
```

6.3 VXLAN Tunnel Not Forming

Symptom:

```bash
Leaf1# show vxlan tunnel
VTEP Name      : vtep1
SIP            : 10.0.0.11
Creation Source: CLI

No tunnels found
```

Diagnosis:

```bash
# 1. Verify EVPN routes are received
Leaf1# show bgp l2vpn evpn route type imet
(should show Type-3 routes from other leaves)

# 2. Check kernel interface
admin@Leaf1:~$ ip link show vtep1
(should show VXLAN interface)

# 3. Check logs
admin@Leaf1:~$ tail -f /var/log/syslog | grep -i vxlan
admin@Leaf1:~$ docker logs bgp | grep -i evpn
```

Common Issues:

- EVPN Type-3 routes not being received
- Loopback not reachable (verify with `ping source-interface Loopback0`)
- vxlanmgr container not running
- Incorrect VNI configuration

6.4 MAC Learning Not Working

Symptom:

```bash
Leaf1# show vxlan remotemac all
No remote MAC entries found
```

Diagnosis:

```bash
# 1. Check if traffic is flowing on access ports
Leaf1# show interface counters | include Ethernet16
(should show RX packets)

# 2. Verify local MACs are learned
Leaf1# show mac address-table
(should show local MACs on access ports)

# 3. Check EVPN Type-2 routes
Leaf1# show bgp l2vpn evpn route type macip
(should show MAC/IP routes)

# 4. Verify FRRouting is advertising local MACs
admin@Leaf1:~$ sudo vtysh -c "show evpn mac vni all"
```

Fixes:

- Ensure hosts are sending traffic
- Verify VLAN configuration on access ports
- Check that bridge learning is enabled
- Verify BGP EVPN session is established

6.5 Asymmetric Traffic Flow

Symptom: Traffic works in one direction but not the other

Diagnosis:

```bash
```

```
# 1. Trace packet path
Leaf1# show mac address-table address 00:11:22:33:44:55
Vlan    MAC Address       Type      Port
----    -----------       ----      ----
10      00:11:22:33:44:55 EVPN      vtep1

# 2. Verify remote VTEP reachability
Leaf1# ping 10.0.0.12 source-interface Loopback0
PING 10.0.0.12 (10.0.0.12) from 10.0.0.11: 56 data bytes
64 bytes from 10.0.0.12: icmp_seq=1 ttl=63 time=1.2 ms

# 3. Check for MTU issues
Leaf1# ping 10.0.0.12 size 9000 df-bit
```

Common Causes:

- MTU mismatch (VXLAN adds 50 bytes overhead)
- Asymmetric ECMP hashing
- One-way reachability in underlay
- MAC not properly synchronized on remote leaf

6.6 Performance Issues

Diagnosis:

```bash
# 1. Check for packet drops
Leaf1# show interface counters errors

# 2. Monitor CPU usage
admin@Leaf1:~$ top
admin@Leaf1:~$ docker stats

# 3. Check ASIC buffer usage
admin@Leaf1:~$ show queue counters

# 4. Verify ECMP is working
Leaf1# show ip route 10.0.0.12
(should show multiple paths)
```

Optimizations:

- Enable hardware VXLAN encap/decap offload
- Tune BGP timers for faster convergence
- Implement QoS policies if needed
- Monitor for micro-bursts causing drops

6.7 Useful Debug Commands

```bash
# Enable BGP debugging
admin@Leaf1:~$ sudo vtysh -c "debug bgp updates"
admin@Leaf1:~$ sudo vtysh -c "debug bgp zebra"

# Enable EVPN debugging
admin@Leaf1:~$ sudo vtysh -c "debug bgp evpn mh events"
admin@Leaf1:~$ sudo vtysh -c "debug zebra vxlan"

# Check Redis database
admin@Leaf1:~$ redis-cli -n 0 KEYS "*"  # APPL_DB
admin@Leaf1:~$ redis-cli -n 4 KEYS "*"  # CONFIG_DB
admin@Leaf1:~$ redis-cli -n 6 KEYS "*"  # STATE_DB

# Monitor syslog
admin@Leaf1:~$ tail -f /var/log/syslog

# Packet capture
admin@Leaf1:~$ sudo tcpdump -i Ethernet0 -w /tmp/capture.pcap
admin@Leaf1:~$ sudo tcpdump -i vtep1 port 4789 -vvv
```

---

7. Advanced Configuration Examples

7.1 Anycast Gateway Configuration

Configure the same gateway IP on all leaves for seamless VM mobility:

```bash
```

```
Leaf1(config)# interface Vlan 10
Leaf1(config-if-Vlan10)# ip address 192.168.10.1/24
Leaf1(config-if-Vlan10)# ip virtual-router address 192.168.10.1
Leaf1(config-if-Vlan10)# ip virtual-router mac-address 00:00:5e:00:01:01

# Repeat same config on Leaf2 and Leaf3
# All leaves answer ARP for 192.168.10.1
```

7.2 ARP Suppression

Reduce BUM traffic by suppressing ARP requests:

```bash
Leaf1(config)# interface vxlan vtep1
Leaf1(config-if-vxlan-vtep1)# map vlan 10 vni 10010
Leaf1(config-if-vxlan-vtep1)# neigh-suppress
```

7.3 VRF Configuration with L3 VNI

For inter-VNI routing:

```bash
Leaf1(config)# ip vrf TENANT1
Leaf1(config-vrf)# vni 50001

Leaf1(config)# interface Vlan 10
Leaf1(config-if-Vlan10)# ip vrf forwarding TENANT1
Leaf1(config-if-Vlan10)# ip address 192.168.10.1/24

Leaf1(config)# router bgp 65001
Leaf1(config-router-bgp)# vrf TENANT1
Leaf1(config-router-bgp-vrf)# address-family l2vpn evpn
Leaf1(config-router-bgp-vrf-af)# advertise ipv4 unicast
```

---

8. Best Practices

Configuration Best Practices

1. **Use consistent naming**: vtep1, Vlan10, etc.

2. **MTU**: Always set MTU ≥ 9100 on all fabric links
3. **BGP timers**: Use BFD for sub-second failure detection
4. **Route reflectors**: For large fabrics (>20 leaves), use spine route reflectors
5. **Monitoring**: Deploy continuous monitoring of BGP sessions and tunnel status

Operational Best Practices

1. **Change management**: Test configurations in lab before production
2. **Backup configs**: `show running-config` to version control
3. **Logging**: Centralize syslog for all switches
4. **Documentation**: Maintain topology diagrams and IP address sheets
5. **Automation**: Use Ansible/Python for consistent deployments

Security Considerations

1. **Management access**: Use dedicated management VRF
2. **BGP authentication**: Configure MD5 passwords for BGP peers
3. **Control plane ACLs**: Restrict access to BGP/SSH ports
4. **VXLAN security**: Consider MACSec for fabric encryption

---

9. Summary

This tutorial covered:

- **SONiC Architecture**: Container-based, modular design with Redis database
- **EVPN-VXLAN**: Modern data center overlay technology
- **Configuration**: Complete 5-router Clos fabric with BGP underlay and EVPN overlay
- **Verification**: Comprehensive show commands at CLI, kernel, and ASIC levels
- **Troubleshooting**: Common issues and resolution steps

SONiC's open-source nature, combined with EVPN-VXLAN, provides a scalable, vendor-neutral solution for modern data center networks. The separation of control plane (BGP EVPN) and data plane (VXLAN) enables flexible network designs with seamless layer 2 extension across layer 3 boundaries.

For production deployments, always test thoroughly, implement proper monitoring, and maintain detailed documentation of your network design and configurations.